

NLP RETRIEVAL PIPELINE

Czech News Corpus Search

Full-text search and retrieval pipeline over 1,000 Czech news articles.

THE ASSIGNMENT

NLP task

Goals:

1. Solution design

Design and implement a basic **retrieval pipeline** for searching in a corpus of news articles.

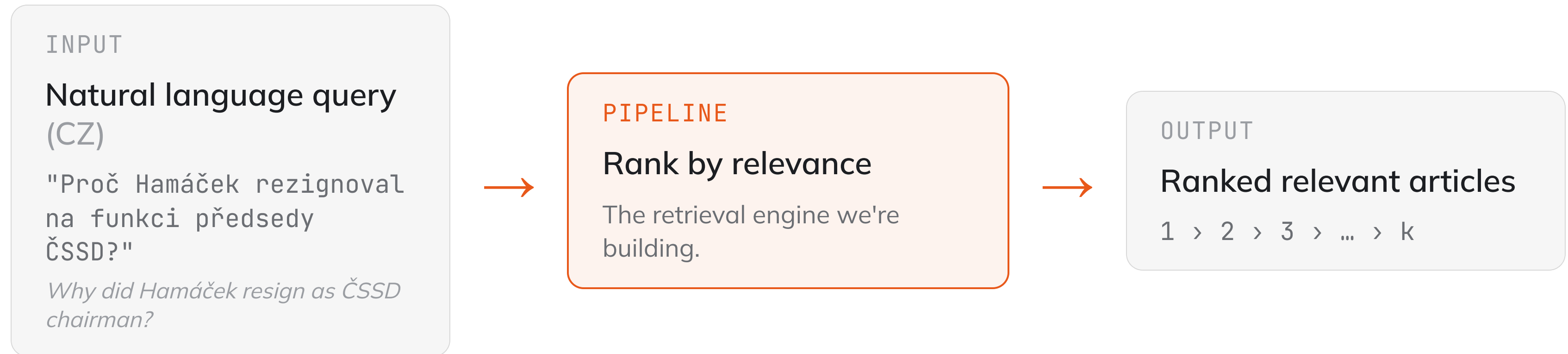
- Build a **full-text search** mechanism, including relevant NLP techniques.
- Focus on high-quality search and retrieval functionality.
- Design and describe the **evaluation methodology**. How would you assess the system's effectiveness?
- Reflect on the **limitations** of your current implementation and suggest **potential improvements**.

2. Industrialization

Draw an **architecture diagram** how the solution can be designed in **production**.

02 • The Problem

Retrieval is the first layer of every modern RAG system.



THE CHALLENGES OF THIS SPECIFIC CASE

Czech morphology

Praha = Prahy = Praze • 7 cases

No ground truth

zero manual labels available

Length variance

431 to 20,000 characters

A PROBLEM RELEVANT FOR DATASENTICS CLIENTS

Czech bank • Compliance & Risk

- ▶ 10,000+ internal policy documents, never leaving the bank's infrastructure
- ▶ Analysts query in natural language
- ▶ Fully self-hosted pipeline: no data sent to external APIs

03 · Agenda

1

Context

Dataset · State of the art



2

Pipeline

V1 baseline · V2 my additions · Sparse · Dense · Hybrid



3

Evaluation & Results

Ground truth · 3 protocols compared



4

Takeaways

What I learned · Limitations



5

Production

FastAPI · Docker · Milvus

04 · 1,000 Czech news articles · simecek/czech_news

headline

Hamáček oznámil
předsednictvu ČSSD, že
končí ve funkci předsedy

*Jan Hamáček announced
his resignation as ČSSD
party chairman*

brief

Dosluhující vicepremiér a
ministr vnitra Jan Hamáček
oznámil...

*Outgoing deputy PM Jan
Hamáček announced...*

content

431 to 20,000 characters

keywords

Jan hamáček · Rezignace ·
Předseda · Čssd · Volby

Domácí

KEY INSIGHT

- 46× length ratio between shortest and longest article
- High variance forces a chunking strategy
- A single embedding cannot represent a 20,000-character article faithfully

05 · State of the art · my implementation choices

Sparse BM25

Exact term matching · no GPU · fast

Query



Tokenize `.split()`



Score against BM25 index



Ranked articles

WHY BM250kapi?

- ▶ No labeled data to train a learned ranker
- ▶ Consistent terminology in news, exact match works well

Dense embeddings

Semantic similarity · handles morphology

Query



Embed MiniLM-L12



Cosine similarity over chunks



Ranked articles

WHY MiniLM-L12?

- ▶ CPU-only, no GPU needed
- ▶ Natively multilingual, handles Czech out of the box

Hybrid RRF fusion

Parameter-free rank fusion · no score normalization needed

Query



BM25 top-100 + Dense top-100



RRF fusion (k=60)

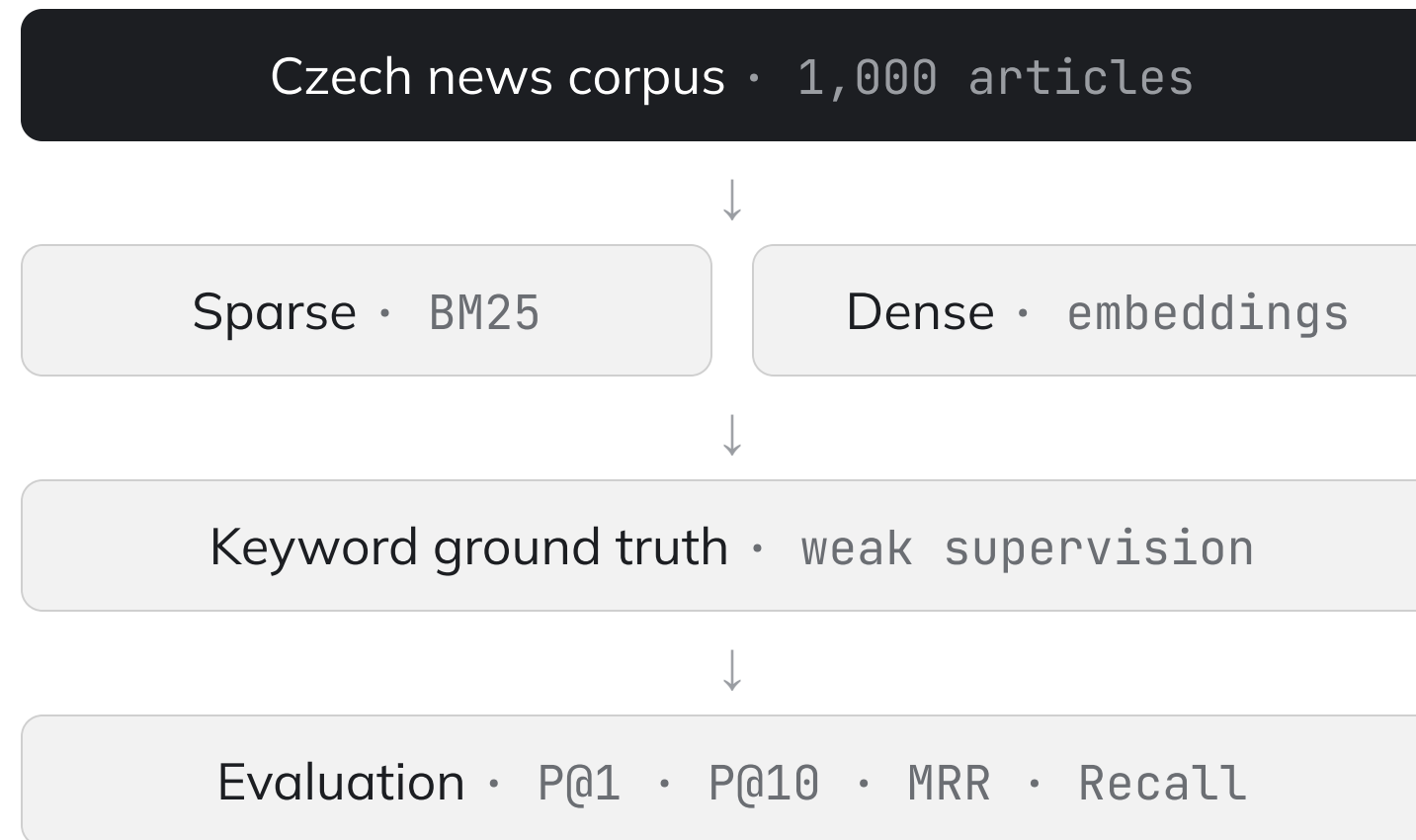


Ranked articles

WHY RRF?

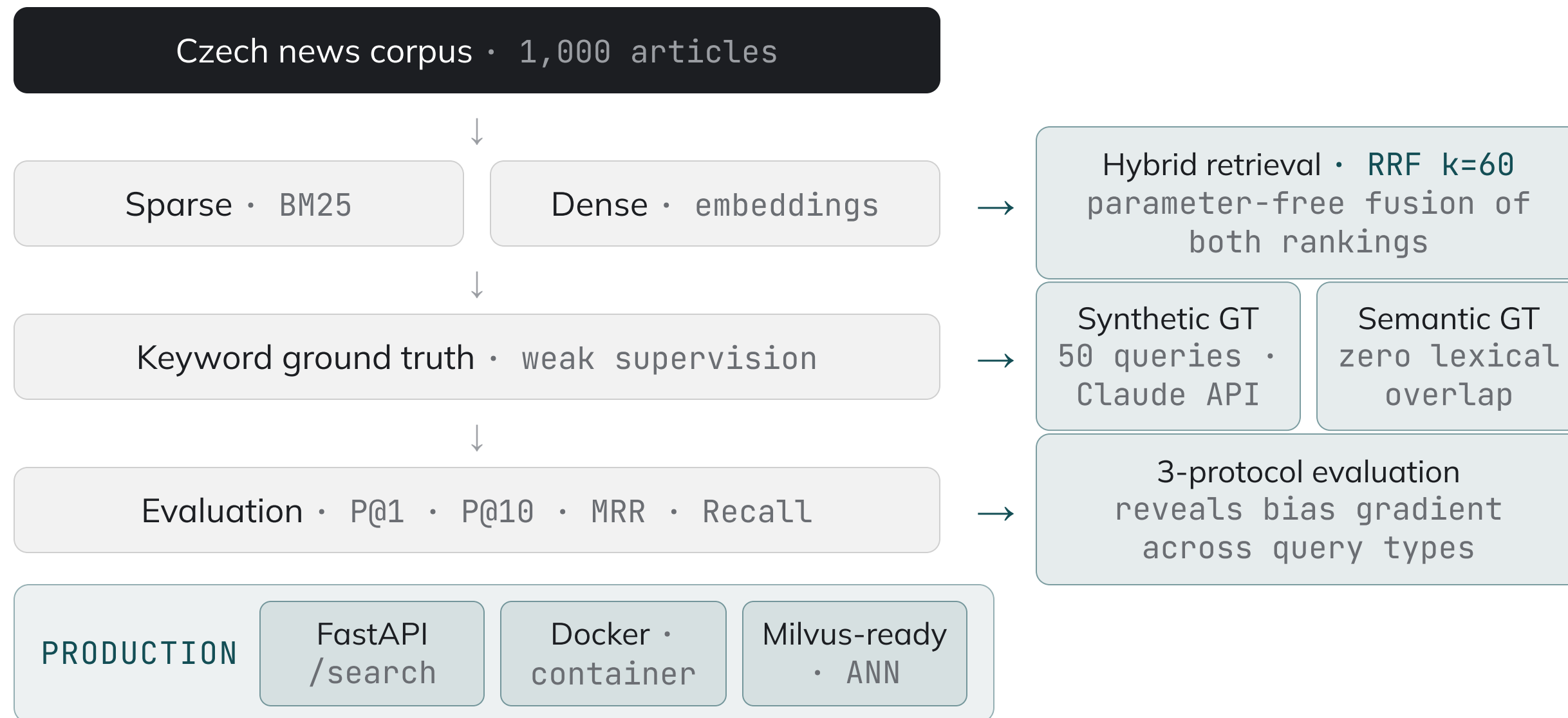
- ▶ Ranks by position, not raw score — no normalization needed
- ▶ Parameter-free. Only k=60

06 · V1 · What a first iteration looks like



■ Charcoal = input data ■ Gray = baseline

07 · V2 · What I added



■ Charcoal = input data ■ Gray = baseline ■ Teal = my additions

08 · Sparse retrieval with BM25

BM25 · ranks by term frequency

01

Rewards exact term matches

02

Weighted against term rarity
across corpus

03

Fast, interpretable, no training
required

WHY BM25 HERE

- ▶ News articles use consistent terminology, exact match works well
- ▶ No labeled data available to train a learned ranker
- ▶ Strong, fast baseline before adding dense retrieval

FIELD WEIGHTING

headline ×2 + **brief** ×2 + **content** ×1

BM25Okapi · tokenized via `.split()`

EXAMPLE

"Hamáček" appears twice

Headline + brief → counted twice → ranks higher

Interpretable

Strong lexical baseline

09 • Embeddings over chunks

Query



Embed (vector)



Cosine similarity



Closest chunks

WHY DENSE HERE

- ▶ BM25 misses morphological variants (Praha, Prahy, Praze)
- ▶ Captures semantic meaning beyond exact term match
- ▶ Complements BM25's lexical blind spots, not a replacement

paraphrase-multilingual-MiniLM-L12-v2

384 dim · 12 layers · no GPU

50+ languages incl. Czech

2,000

characters per chunk

200

character overlap

2,117

chunks from 1,000 articles

Search by chunk,
return full article

COSINE SIMILARITY

Angle between vectors, length-independent

SCALE

Brute-force fine at 2,117 chunks, ANN (Milvus) only needed at scale

10 • Weak supervision from keywords

Built automatically from the `keywords` column. No manual annotation.

STRATEGY 1 precision

Query = **all keywords** of an article

Expect that exact article at rank 1.

"Jan hamáček Rezignace Předseda Čssd Volby"
→ article #0 expected at rank 1

STRATEGY 2 recall

Query = **one keyword**

Expect all articles tagged with that keyword.

"Volby"
→ articles #0, #45, #203 all in top 10

11 • How I measured retrieval quality

KEYWORD GT	SYNTHETIC GT	SEMANTIC GT
SOURCE Article keywords from metadata QUERY STYLE Exact keyword strings BIAS Favors BM25 by construction	SOURCE Claude API generated queries QUERY STYLE Natural language, topic-based BIAS May reflect LLM patterns	SOURCE Claude API, zero lexical overlap enforced QUERY STYLE Paraphrase-style, no shared words BIAS Most honest. Closest to real users

50 queries per protocol • ground truth = source article • metric = P@1 and MRR@10

12 • Results • 3 protocols compared

Methodology: keyword weak supervision → LLM-generated queries → lexically-independent semantic queries

METRIC	Sparse (BM25)	Dense	Hybrid (RRF)
Keyword GT · P@1	0.62	0.25	0.43
Keyword GT · MRR@10	0.69	0.32	0.53
Synthetic GT · P@1	0.74	0.64	0.74
Synthetic GT · MRR@10	0.80	0.69	0.82
Semantic GT · P@1	0.44	0.48	0.48
Semantic GT · MRR@10	0.53	0.59	0.58

150 queries • 50 per protocol • Claude API claude-sonnet-4-6

KEY INSIGHT

- ▶ Keyword GT → BM25 wins (expected: queries match keywords exactly)
- ▶ Synthetic GT → Hybrid leads on MRR
- ▶ Semantic GT → Dense & Hybrid **outperform BM25**
- ▶ Hybrid = most stable across all query types

⚠ **Limitations:** 50 queries per protocol; 4-point differences may not be significant (no bootstrap CIs), and Claude-generated queries may carry their own bias.

IN PRODUCTION Hybrid search combining both. BM25 for precision on known terms, dense for semantic recall.

13 • Honest limitations • what I'd build next

LIMITATIONS

Evaluation robustness

- ▶ 50 queries per protocol: statistically weak
- ▶ A 4-point gap is likely not significant
- ▶ No bootstrap confidence intervals

Synthetic GT bias

- ▶ Claude-generated queries may reflect LLM patterns
- ▶ Cloud US API vs sovereign-deployment tension

No human ground truth

- ▶ No manual relevance judgments
- ▶ Weak supervision + synthetic labels only
- ▶ Priority fix: 100–200 annotated queries

Scalability

- ▶ Brute-force cosine breaks at 100M articles
- ▶ Storage, indexing and latency all explode
- ▶ Motivates Milvus + ANN

WHAT I'D BUILD NEXT

RETRIEVAL QUALITY

- ▶ **Czech lemmatization**: cut morphological noise for BM25
- ▶ **bge-reranker-v2-m3**: cross-encoder on top-20, self-hosted

EVALUATION

- ▶ **Human-labeled benchmark**: 100–200 queries
- ▶ **Statistical validation**: bootstrap CIs

SCALE

- ▶ **Milvus + ANN**: replace brute-force cosine
- ▶ **Distributed indexing**: handle corpus growth

PRODUCTION MONITORING

- ▶ **Online signals**: CTR, click position, dwell time
- ▶ **A/B testing**: compare versions on real traffic
- ▶ **Feedback loop**: build a human-labeled set over time

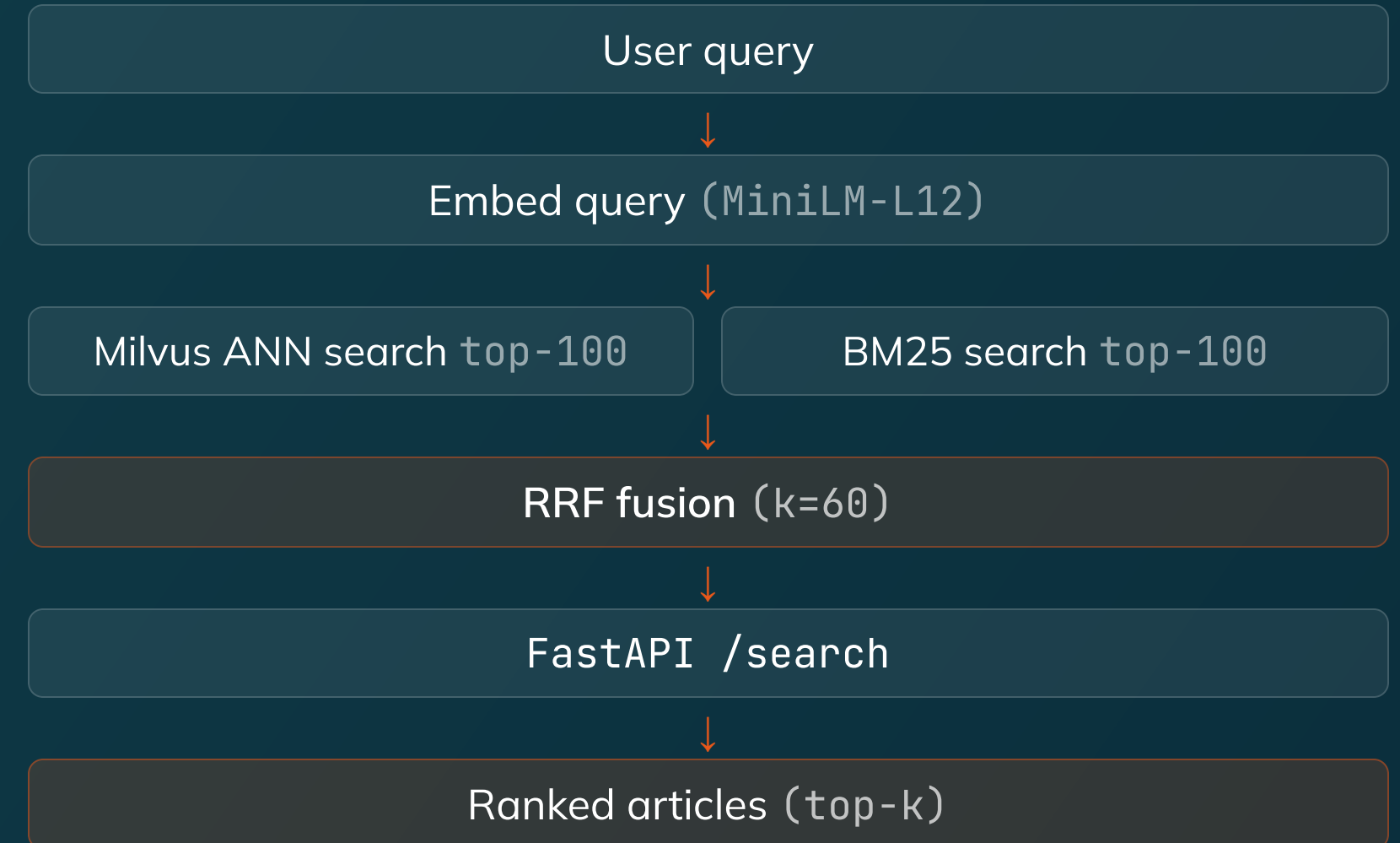
14 • From prototype to deployed service

This architecture serves as the retrieval layer of an enterprise RAG system.

INGESTION • OFFLINE



QUERY PATH • REAL-TIME



Milvus

self-hosted vector store • ANN search at scale

FastAPI + Uvicorn

always-on REST server

RRF k=60

parameter-free fusion • no score normalization

RUNTIME BM25 indexing ~2.3 s Embedding generation (CPU) ~3.9 min Query latency ~62 ms

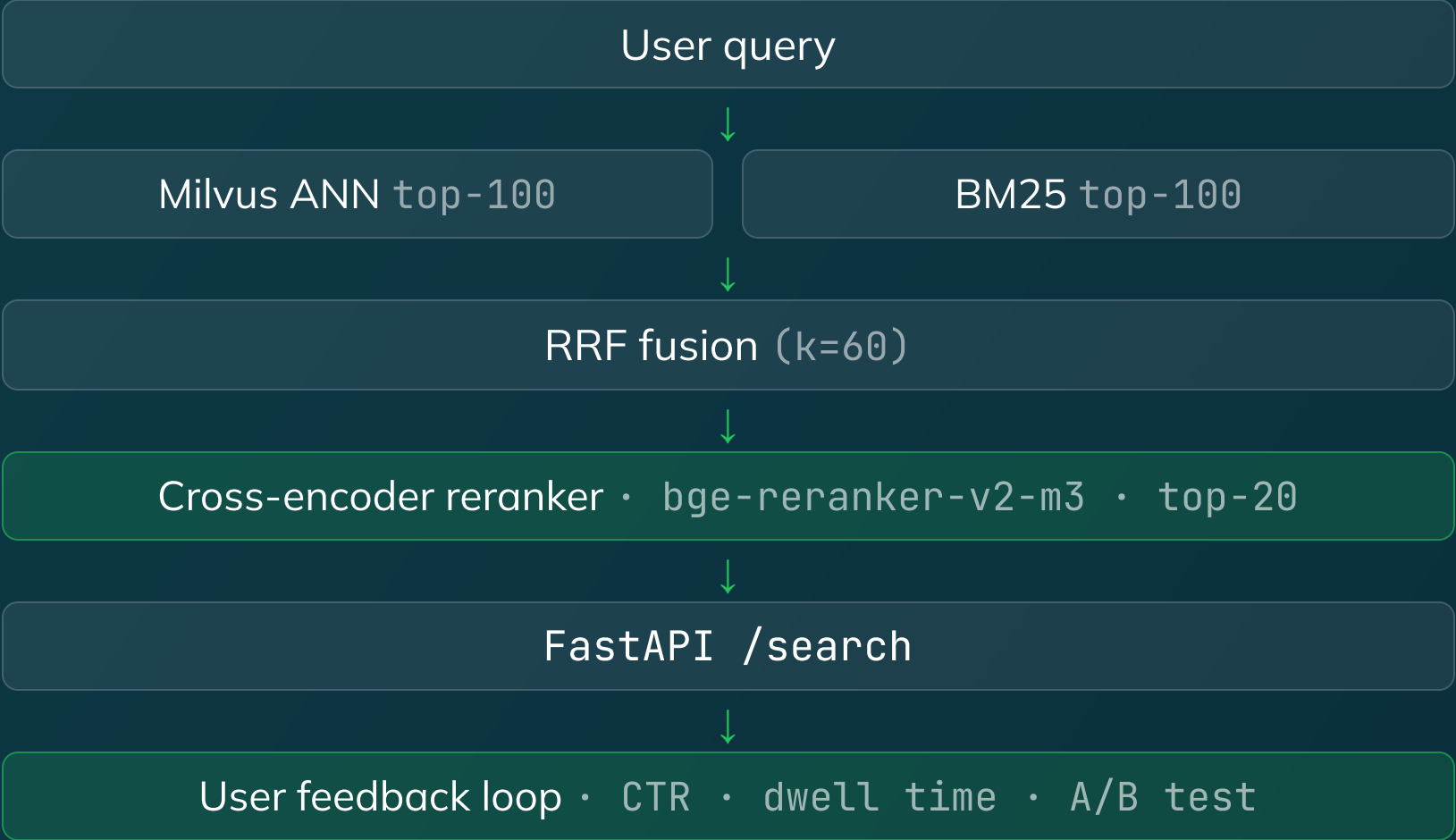
► Deployed via Docker • FastAPI serves the hybrid pipeline on port 8000 • embeddings mounted as volume

What the pipeline looks like with improvements

INGESTION · OFFLINE



QUERY PATH · REAL-TIME

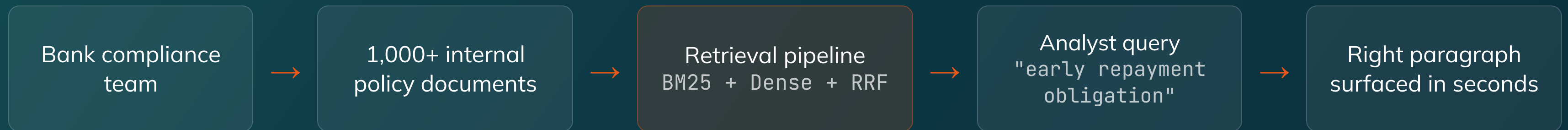


► bge-reranker-v2-m3: open-source, multilingual, self-hosted. Consistent with sovereign deployment approach

Gray = current Green = proposed improvements

15 • From news retrieval to enterprise RAG

This pipeline is the retrieval layer DataSentics builds for clients like this.



THE THROUGH-LINE

The retrieval challenges in this assignment, morphology, no ground truth, length variance, are the same ones DataSentics clients face daily. This project demonstrates the ability to build, evaluate honestly, and iterate on exactly this kind of system.

Let's discuss

QUESTIONS I'D LOVE YOUR PERSPECTIVE ON

- ▶ How do you evaluate retrieval quality in production without human labels?
- ▶ Do your clients accept synthetic evaluation as a proxy?
- ▶ When do you introduce rerankers in a production pipeline?
- ▶ How important is model sovereignty when selecting embedding models?

WHAT THIS PROJECT SHOWED ME

- ▶ Evaluation design is as important as retrieval design
- ▶ Engineering pragmatism: ship the full pipeline first, optimize components second
- ▶ Czech morphology is a real constraint. Lemmatization would be my first retrieval upgrade